

This member-only story is on us. [Upgrade](#) to access all of Medium.

★ Member-only story

Support Vector Machine(with Numerical Example)



Balaji C · [Follow](#)

7 min read · Jan 19, 2023



17



1

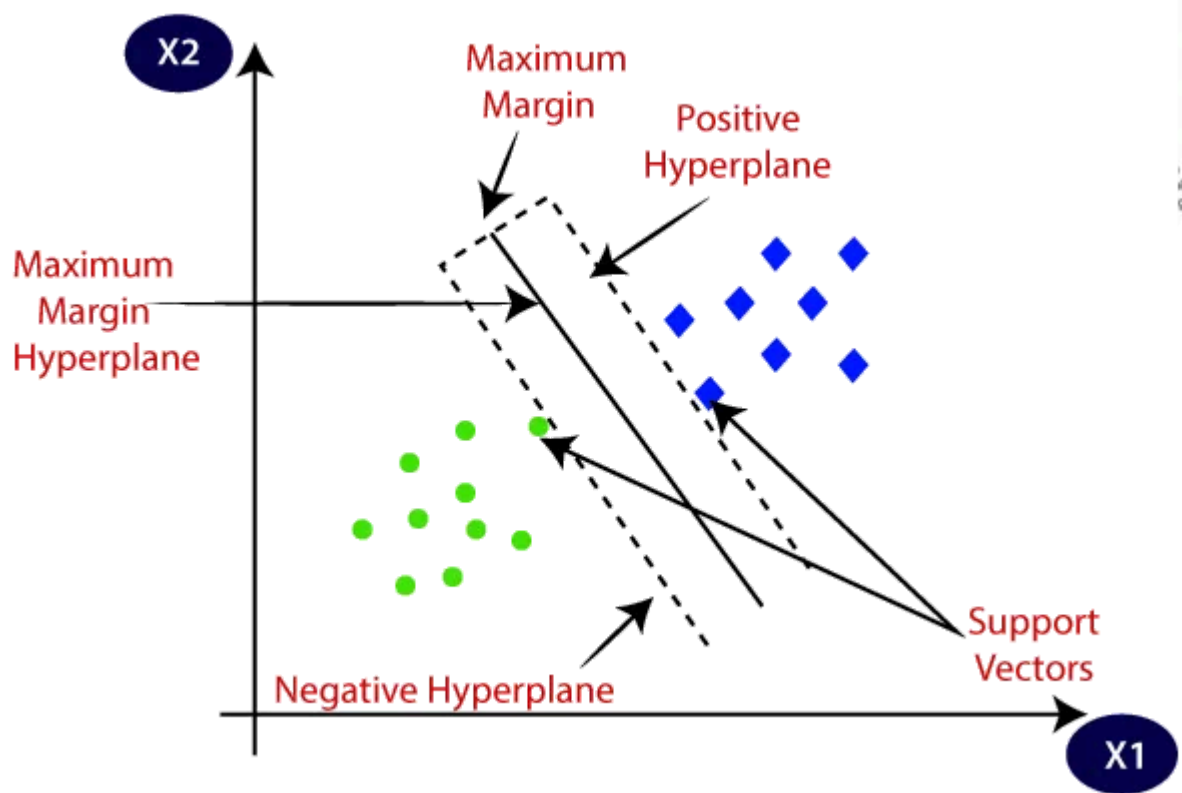


SVM is a one of the most popular supervised machine learning algorithm, which can be used for both classification and regression but mainly used in area of classification.

Main goal of SVM is to create best fit line or boundary that can segregate n-dimensional space into classes, so that we can put the data point in correct category for future prediction.

The best decision boundary is called **hyper-plane**.

SVM chooses the extreme points/vectors that helps in creating a hyper-plane. These extreme cases are called as **support vectors**. Hence algorithm is termed as support vector machine.



Linear SVM

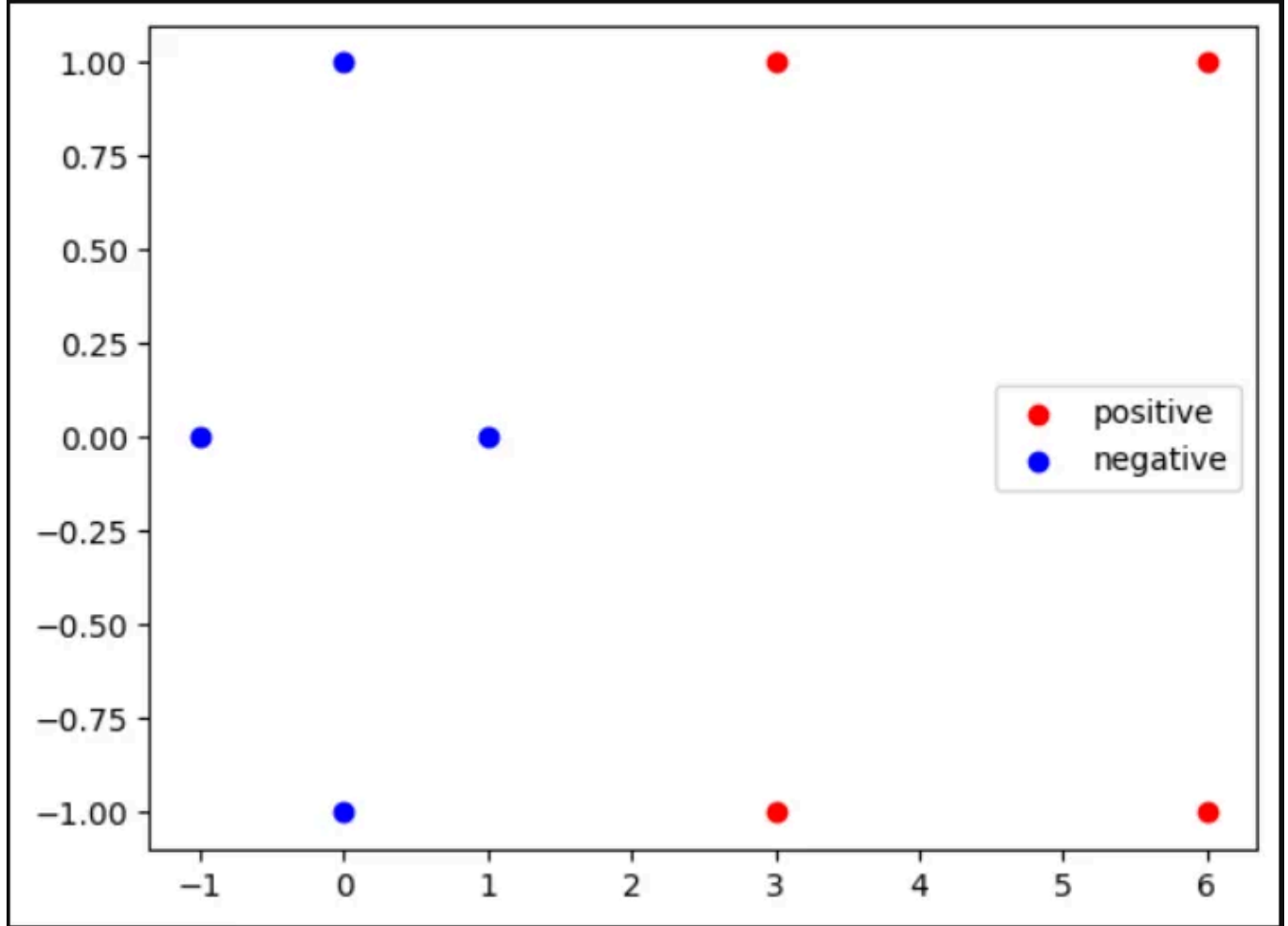
There are mainly two types of SVM

1. **Linear SVM:** Linear SVM is used for linearly separable data, which means if a dataset can be classified into two classes by using a single straight line, then such data is termed as linearly separable data, and classifier is used called as Linear SVM classifier.

Numerical example for Linear SVM:

Q. Positively labelled data points (3,1)(3,-1)(6,1)(6,-1) and Negatively labelled data points (1,0)(0,1)(0,-1)(-1,0)

Solution: for all negative labelled output is -1 and for all positive labelled output is 1.



graph for above table

Now adding 1 to all points

$$s_1 = (3,1) \Rightarrow s_1' = (3,1,1) \mid s_2 = (3,-1) \Rightarrow s_2' = (3,1,-1)$$

$$s_3 = (6,1) \Rightarrow s_3' = (6,1,1) \mid s_4 = (6,-1) \Rightarrow s_4' = (6,1,-1)$$

$$s_5 = (1,0) \Rightarrow s_5' = (1,0,1) \mid s_6 = (0,1) \Rightarrow s_6' = (0,1,1)$$

$$s_7 = (0,-1) \Rightarrow s_7' = (0,-1,1) \mid s_8 = (-1,0) \Rightarrow s_8' = (-1,0,1)$$

from the graph we can see there is one negative point $\alpha_1 = (1,0,1)$ and two positive points $\alpha_2 = (3,1,1)$ & $\alpha_3 = (3,-1,1)$ form support vectors

Generalized equation

$$\alpha_1 * s_1' * s_1' + \alpha_2 * s_1' * s_2' + \alpha_3 * s_1' * s_3' = -1 \rightarrow 1$$

$$\alpha_1 * s_2' * s_1' + \alpha_2 * s_2' * s_2' + \alpha_3 * s_2' * s_3' = 1 \rightarrow 2$$

$$\alpha_1 * s_3 + \alpha_2 * s_3 + \alpha_3 * s_3 = 1 \rightarrow 3$$

On solving these equations

$$\text{eq 1} \Rightarrow \alpha_1 * (1,0,1) + \alpha_2 * (1,0,1) + \alpha_3 * (1,0,1) = -1$$

$$\text{eq 2} \Rightarrow \alpha_1 * (3,1,1) + \alpha_2 * (3,1,1) + \alpha_3 * (3,1,1) = 1$$

$$\text{eq 3} \Rightarrow \alpha_1 * (3,-1,1) + \alpha_2 * (3,-1,1) + \alpha_3 * (3,-1,1) = 1$$

$$2\alpha_1 + 4\alpha_2 + 4\alpha_3 = -1 \rightarrow 4$$

$$4\alpha_1 + 11\alpha_2 + 9\alpha_3 = 1 \rightarrow 5$$

$$4\alpha_1 + 9\alpha_2 + 11\alpha_3 = 1 \rightarrow 6$$

on solving these equations 4,5 and 6 we get,

$$\alpha_1 = -3.5, \alpha_2 = 0.75 \text{ and } \alpha_3 = 0.75$$

To find hyper-plane

$$W = \sum \alpha_i * s_i$$

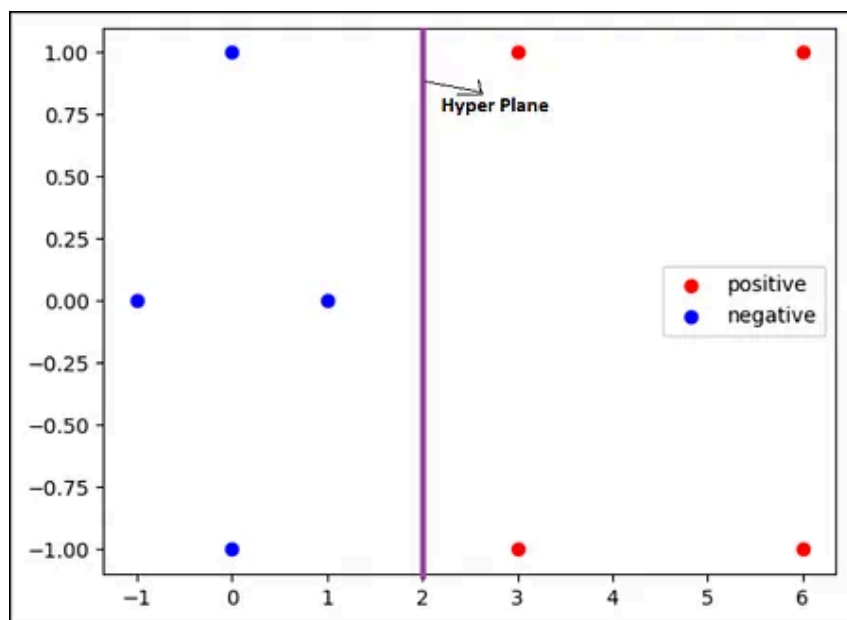
$$W = -3.5 * (1,0,1) + 0.75 * (3,1,1) + 0.75 * (3,-1,1)$$

$$W = (1,0,-2)$$

$$y = W^T x + b$$

$W^T = (1,0)$ and $b = 2$

so the best fit line or hyper plane is at $(0,2)$ which splits the data points into two classes



Final Graph for linear SVM

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import svm, datasets

iris = datasets.load_iris()
X = iris.data[:, :2]
y = iris.target

C = 1.0 # SVM regularization parameter
svc = svm.SVC(kernel='linear', C=1, gamma=0).fit(X, y)

x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
h = (x_max / x_min)/100
xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
    np.arange(y_min, y_max, h))

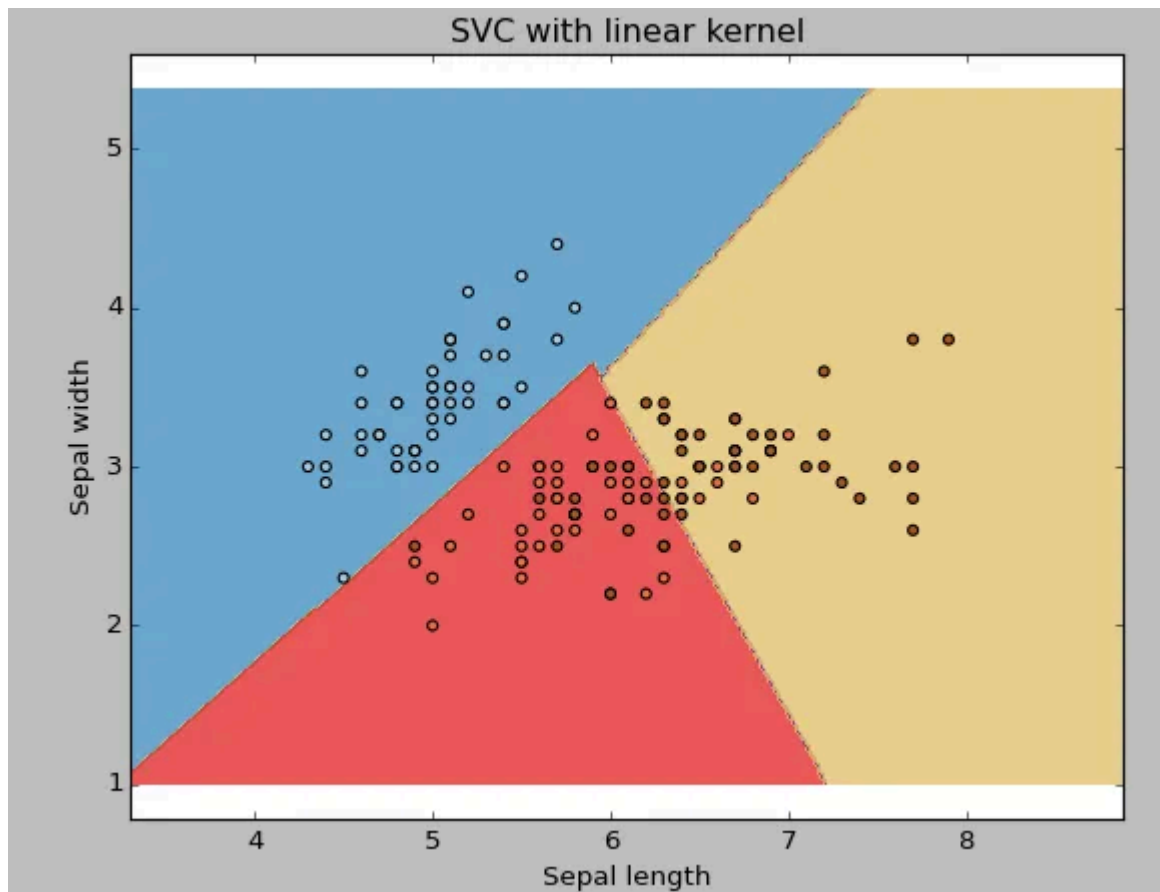
plt.subplot(1, 1, 1)
Z = svc.predict(np.c_[xx.ravel(), yy.ravel()])
```

```

Z = Z.reshape(xx.shape)
plt.contourf(xx, yy, Z, cmap=plt.cm.Paired, alpha=0.8)

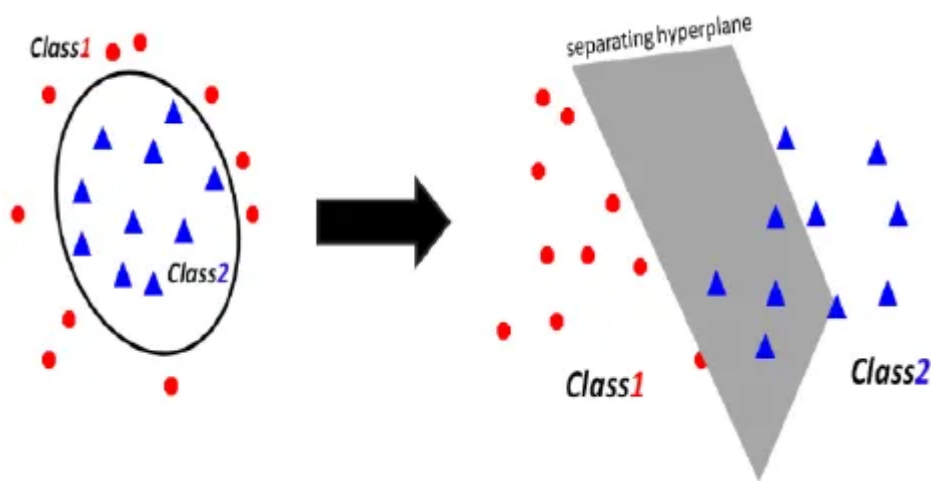
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Paired)
plt.xlabel('Sepal length')
plt.ylabel('Sepal width')
plt.xlim(xx.min(), xx.max())
plt.title('SVC with linear kernel')
plt.show()

```



2. Non-linear SVM: Non-Linear SVM is used for non-linearly separated data, which means if a dataset cannot be classified by using a straight line, then such data is termed as non-linear data and classifier used is called as Non-linear SVM classifier.

In non-linear SVM we will use Kernel SVM to convert low dimensional data into high dimensional data so that small hyper plane will be created.



Numerical example for Non-Linear SVM:

Q. Positively labelled data points (2,2)(2,-2)(-2,-2)(-2,2) and Negatively labelled data points (1,1)(1,-1)(-1,-1)(-1,1)

Solution: We have to find a hyper-plane i.e it divides data into two different classes. But this is of type non-linear SVM so we need to convert data from one feature space to another feature space using Kernel SVM.

Kernel SVM Condition:

$$\Phi_1 \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{cases} \begin{pmatrix} 4 - x_2 + |x_1 - x_2| \\ 4 - x_1 + |x_1 - x_2| \end{pmatrix} & \text{if } \sqrt{x_1^2 + x_2^2} > 2 \\ \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} & \text{otherwise} \end{cases}$$

(1) For positive labelled data:

$$\theta(2,2) \Rightarrow (\sqrt{2^2 + 2^2}) = (\sqrt{8}) > 2$$

$$\text{so } \theta(x_1) = 4 - 2 + |2 - 2| \Rightarrow \theta(x_1) = 2 \text{ and } \theta(x_2) = 4 - 2 + |2 - 2| \Rightarrow \theta(x_2) = 2$$

$$\theta(2,-2) \Rightarrow (\sqrt{2^2+2^2}) = (\sqrt{8}) > 2$$

$$\text{so } \theta(x_1) = 4 + 2 + |2 + 2| \Rightarrow \theta(x_1) = 10 \text{ and } \theta(x_2) = 4 - 2 + |2 + 2| \Rightarrow \theta(x_2) = 6$$

$$\theta(-2,-2) \Rightarrow (\sqrt{2^2+2^2}) = (\sqrt{8}) > 2$$

$$\text{so } \theta(x_1) = 4 + 2 + |-2 + 2| \Rightarrow \theta(x_1) = 6 \text{ and } \theta(x_2) = 4 + 2 + |-2 + 2| \Rightarrow \theta(x_2) = 6$$

$$\theta(-2,2) \Rightarrow (\sqrt{2^2+2^2}) = (\sqrt{8}) > 2$$

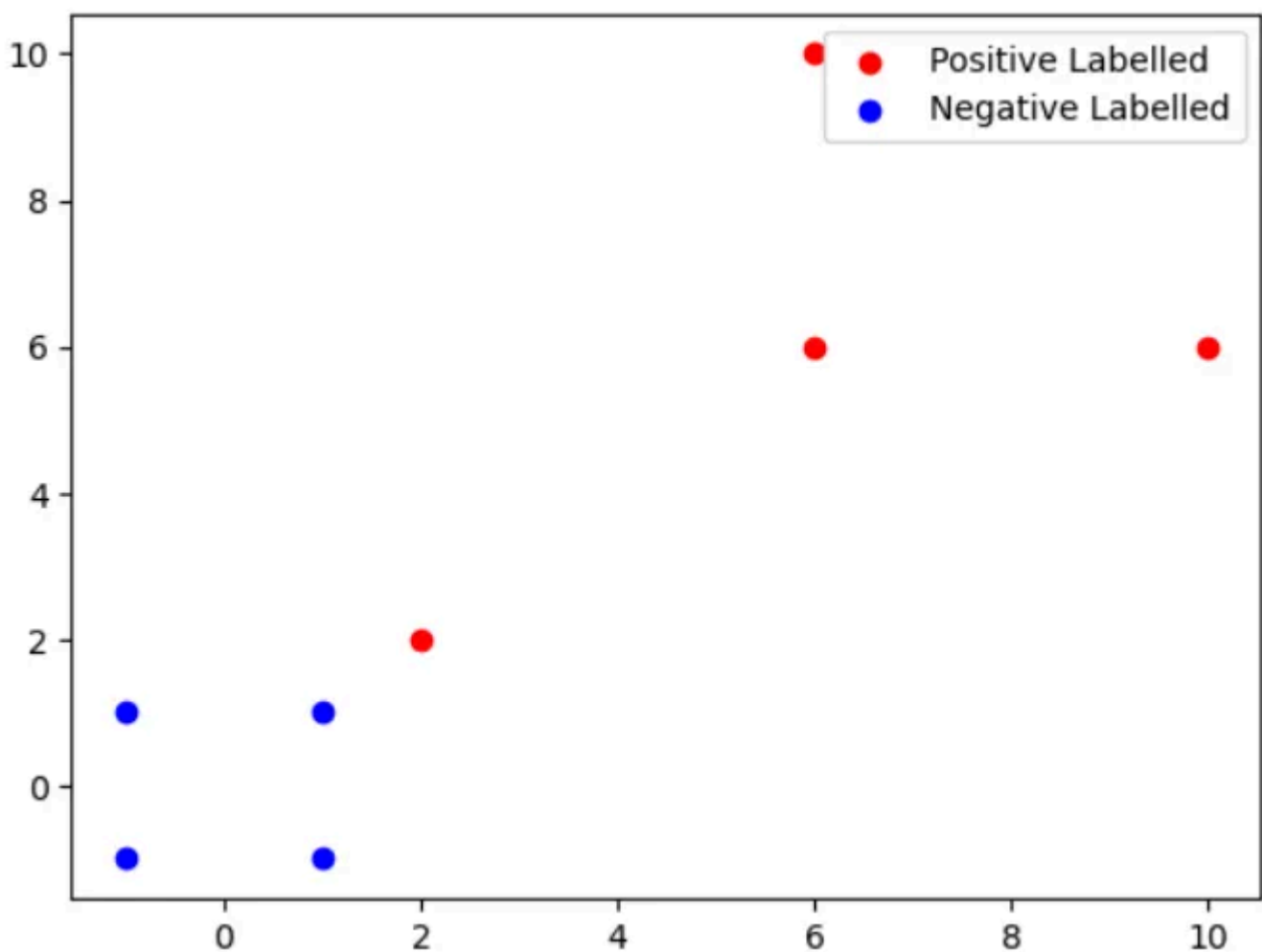
$$\text{so } \theta(x_1) = 4 - 2 + |-2 - 2| \Rightarrow \theta(x_1) = 6 \text{ and } \theta(x_2) = 4 + 2 + |-2 - 2| \Rightarrow \theta(x_2) = 10$$

Hence overall positive labelled data is rechanged as (2,2), (10,6), (6,6), (6,10)

(2) For negative labelled data points:

Here all values are (1, 1) only thing is negative and positive sign changes, so if we square root them then all values will be less than 2. Finally there is no change in the points.

Final plot is



From the graph we can say support vectors are $S_1 = (1,1)$ and $S_2 = (2,2)$ so adding 1 to support vectors, $S_1' = (1,1,1)$ and $S_2' = (2,2,1)$.

$$\alpha_1 * s_1' * s_1' + \alpha_2 * s_1' * s_2' = -1 \rightarrow 1$$

$$\alpha_1 * s_2' * s_1' + \alpha_2 * s_2' * s_2' = 1 \rightarrow 2$$

$$\alpha_1 * (1,1,1) * (1,1,1) + \alpha_2 * (1,1,1) * (2,2,1) = -1 \rightarrow 1$$

$$\alpha_1 * (2,2,1) * (1,1,1) + \alpha_2 * (2,2,1) * (2,2,1) = 1 \rightarrow 2$$

$$3\alpha_1 + 5\alpha_2 = -1 \rightarrow 1 \text{ and } 5\alpha_1 + 9\alpha_2 = 1 \rightarrow 2$$

On solving equations $\alpha_1 = -7$ and $\alpha_2 = 4$

To find hyper-plane

$$W' = \sum \alpha_i * s_i$$

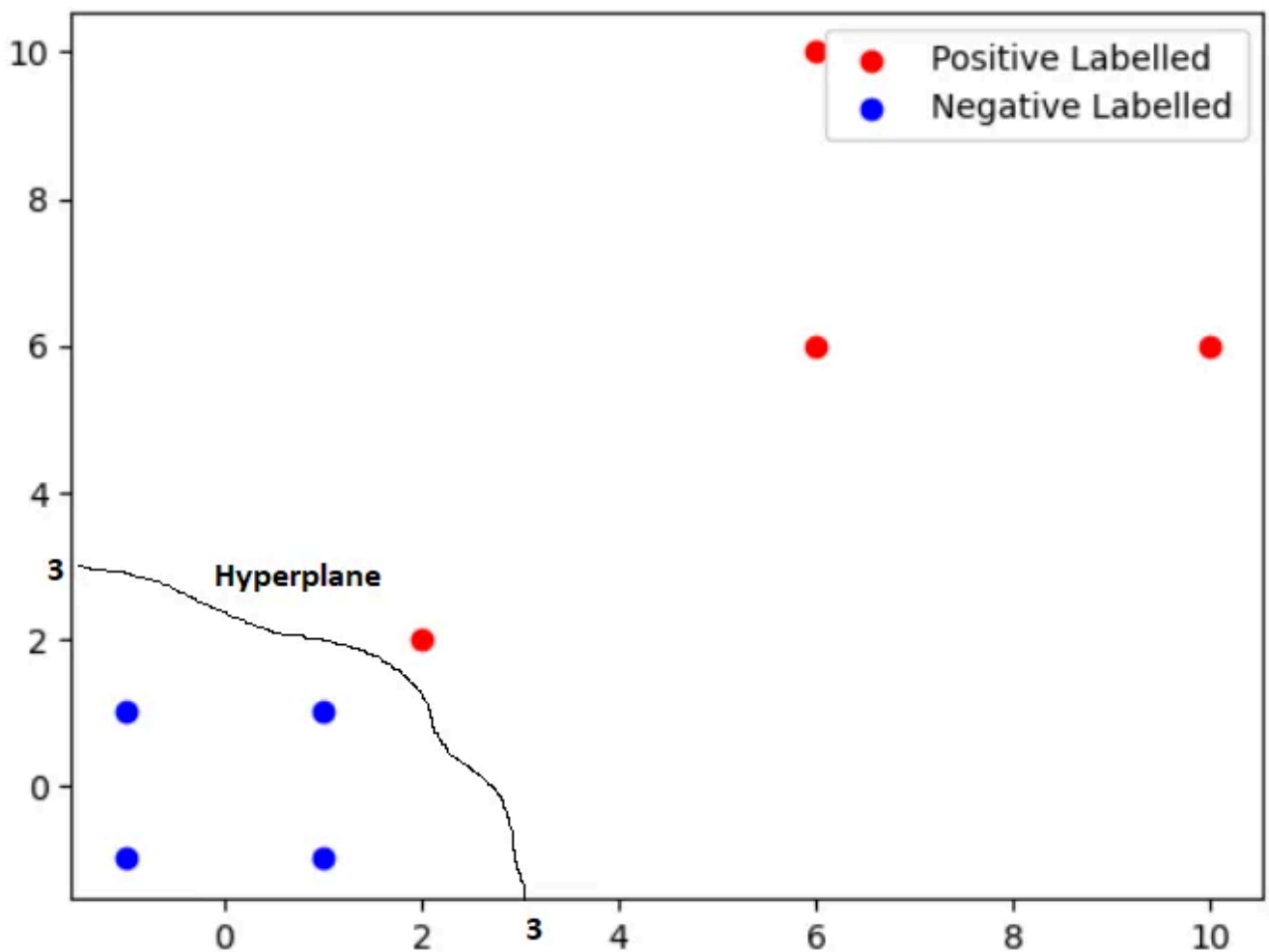
$$W' = -7 * (1,1,1) + 4 * (2,2,1)$$

$$W' = (1,1,-3)$$

$$y = W'x + b$$

$$W' = 1,1 \text{ and } b = 3$$

on drawing graph



At 3 hyper-plane is drawn.

```
import numpy as np
import pandas as pd
```

```

import matplotlib.pyplot as plt
from matplotlib import style
from sklearn.svm import SVC

style.use('fivethirtyeight')

# create mesh grids
def make_meshgrid(x, y, h = .02):
    x_min, x_max = x.min() - 1, x.max() + 1
    y_min, y_max = y.min() - 1, y.max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
    return xx, yy

# plot the contours
def plot_contours(ax, clf, xx, yy, **params):
    Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)
    out = ax.contourf(xx, yy, Z, **params)
    return out

color = ['r', 'b', 'g', 'k']

iris = pd.read_csv("iris-data.txt").values

features = iris[0:150, 2:4]
level1 = np.zeros(150)
level2 = np.zeros(150)
level3 = np.zeros(150)

# level1 contains 1 for class1 and 0 for all others.
# level2 contains 1 for class2 and 0 for all others.
# level3 contains 1 for class3 and 0 for all others.
for i in range(150):
    if i >= 0 and i < 50:
        level1[i] = 1
    elif i >= 50 and i < 100:
        level2[i] = 1
    elif i >= 100 and i < 150:
        level3[i] = 1

# create 3 svm with rbf kernels
svm1 = SVC(kernel = 'rbf')
svm2 = SVC(kernel = 'rbf')
svm3 = SVC(kernel = 'rbf')

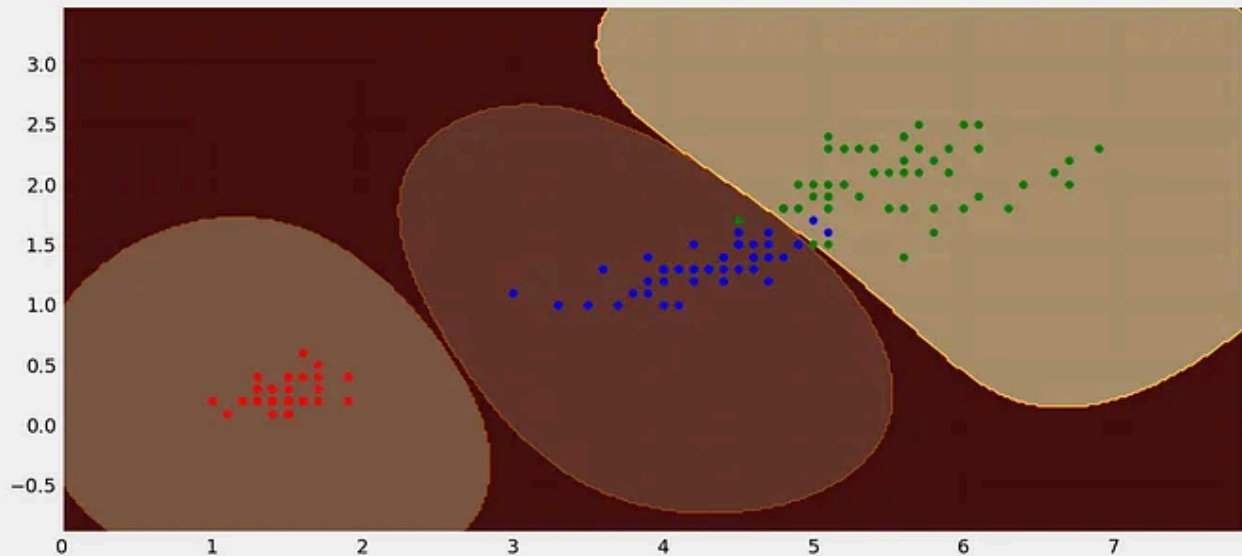
# fit each svm's
svm1.fit(features, level1)
svm2.fit(features, level2)
svm3.fit(features, level3)

fig, ax = plt.subplots()
X0, X1 = iris[:, 2], iris[:, 3]
xx, yy = make_meshgrid(X0, X1)

```

```
color = ['r', 'b', 'g', 'k']

for i in range(len(iris)):
    plt.scatter(iris[i][2], iris[i][3], s = 30, c = color[int(iris[i][4])])
plt.show()
```



Advantages:

- It works really well with a clear margin of separation
- It is effective in high dimensional spaces.
- It is effective in cases where the number of dimensions is greater than the number of samples.
- It uses a subset of training points in the decision function (called support vectors), so it is also memory efficient.

Disadvantages:

- It doesn't perform well when we have large data set because the required training time is higher

- It also doesn't perform very well, when the data set has more noise i.e. target classes are overlapping
- SVM doesn't directly provide probability estimates, these are calculated using an expensive five-fold cross-validation. It is included in the related SVC method of Python scikit-learn library.



Written by Balaji C

67 Followers · 115 Following

Follow

Responses (1)



What are your thoughts?

Respond



Bhuvaneshwarmarri

9 months ago



How to find kernal svm condition



Reply

More from Balaji C

er the given Dataset ,Apply Naive Baye’s Algorithm and Predict that if a
as the following properties then which type of the fruit it is
: {Yellow , Sweet ,long}

ncy Table:

	Yellow	Sweet	Long	Total
	350	450	0	650
a	400	300	350	400
	50	100	50	150
	800	850	400	1200

 Balaji C

Naive Bayes(Numerical Example)

Naive Bayes algorithm is a supervised machine learning algorithm which is based o...

★ Sep 29, 2022 🖱️ 28 💬 3 📌+ ...

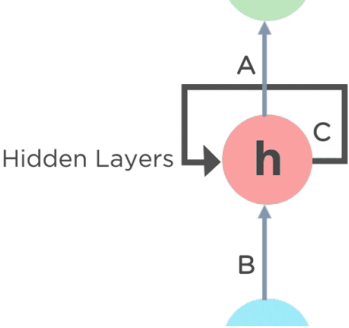
Gender	Height(ft)	Weight(lbs)	FootSize(Inch)
Male	6	180	12
Male	5.92	190	11
Male	5.58	170	12
Male	5.92	165	10
Female	5	100	6
Female	5.5	150	8
Female	5.42	130	7
Female	5.75	150	9

 Balaji C

Gaussian Naive Bayes (Continuous values example)

In this article we see a numerical example for calculating conditional probability for...

★ Jan 12, 2023 🖱️ 2 📌+ ...

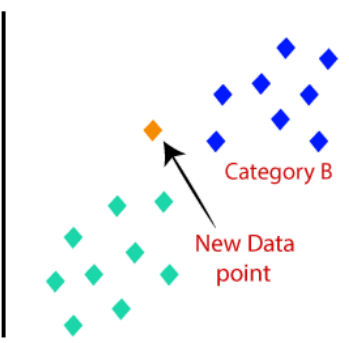


 Balaji C

RNN (Forward and Backward Propagation)

Recurrent Neural Network is a type of neural network in which output from previous step i...

★ Jan 21, 2023 🖱️ 1 📌+ ...



 Balaji C

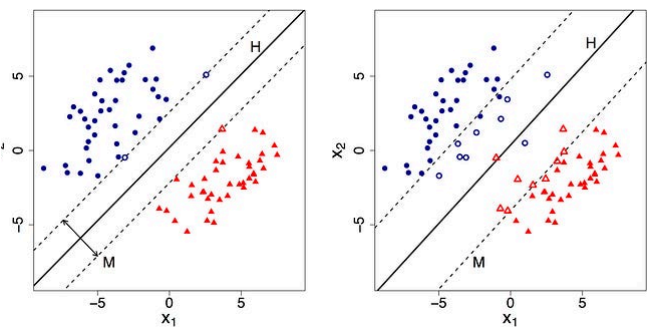
K-Nearest Neighbor(KNN) With Numerical example

KNN is one of the simplest machine learning algorithm for classification and regression...

★ Sep 28, 2022 🖱️ 3 📌+ ...

See all from Balaji C

Recommended from Medium



 Hrishikesh Bele

SVM

Q. What is a Support Vector Machine (SVM)?

Dec 12, 2024



Age		Range	Frequency
27	Encoding Numerical Features	10-15	4
28		16-20	6
39		21-25	10
18		26-30	12
45			

 ajaymehta

“Exploring Different Types of Binning and Discretization...

disclaimer: read it below blog first

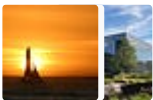
🌟 6d ago 🖱️ 1



Lists



Staff picks
796 stories · 1561 saves



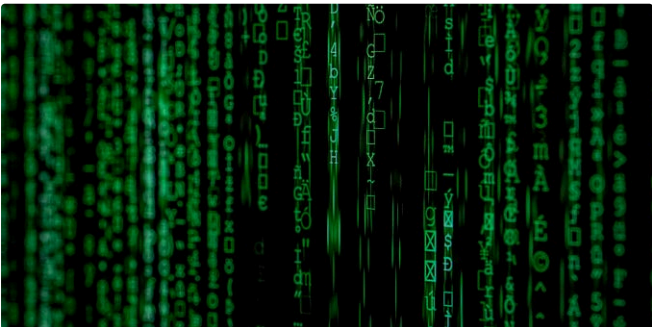
Stories to Help You Level-Up at Work
19 stories · 912 saves



Self-Improvement 101
20 stories · 3191 saves



Productivity 101
20 stories · 2706 saves



Mathematics Behind Support Vector Machines (SVM) Algorithm

A Detailed Guide to Understand the Mathematics Behind SVM

★ Jul 24, 2024 🖱️ 77 💬 1 📖 + ...



 AI SageScribe

Constructing a Multilayer Perceptron (MLP) from Scratch in...

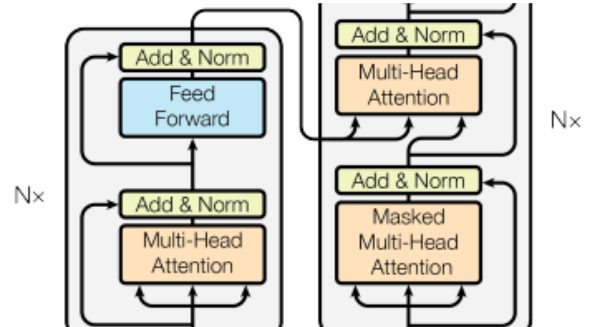
We'll dive into the implementation of a basic neural network in Python, without using any...

★ Jul 14, 2024 📖 + ...

Image Feature Extraction using Python - Part I

Basics of Image feature extraction techniques using python

★ Sep 5, 2024 🖱️ 312 💬 7 📖 + ...



 ajaymehta

Demystifying Transformers: Components and Coding from...

"Attention Is All You Need" research paper, here : <https://arxiv.org/pdf/1706.03762....>

★ Dec 30, 2024 🖱️ 10 📖 + ...

See more recommendations